

21ST OCT 2025

Designing a digital audio visualiser

using ReactJS, threejs, state
managements, etc

ISSAC TING

3 core layers

React controls:

- Expand/collapse UI
- Showing circular sliders
- Keeping audio state

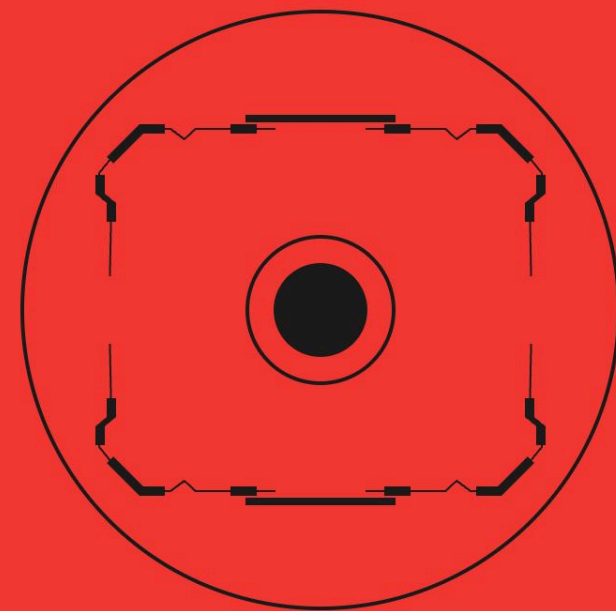
Web Audio API

- Opens the microphone
- Runs real-time frequency analysis
- Extracts:
 - bass
 - mid
 - treble

Components

- SVG
- threejs shaders

Use CSS transitions for smooth state changes.



How It Works:

- When ``.expanded`` class is added, CSS automatically animates
- ``cubic-bezier(0.4, 0, 0.2, 1)`` = "ease-out" timing function
- Browser handles animation (GPU accelerated)

```
const [isExpanded, setIsExpanded] = useState(false);
```


Use CSS transitions for smooth state changes.



```
const [isExpanded, setIsExpanded] = useState(true);
```

```
useEffect(() => {  
  if (isExpanded && !isInitialized) {  
    init();  
  } else if (!isExpanded && isInitialized) {  
    stop();  
  }  
}, [isExpanded, isInitialized, init, stop]);
```

```
const handleExpand = () => {  
  setIsExpanded(true);  
};
```

```
const handleCollapse = () => {  
  setIsExpanded(false);  
};
```

```
<  
div className=  
{`scifi-frame ${isExpanded ? 'expanded' : ''}`}  
>
```

`audioAnalyzer.js`

- Requests microphone access
- Performs FFT analysis (1024 frequency bins)
- Extracts bass (20-250Hz), mid (250-2000Hz), treble (2000-11000Hz)
- Detects beats using statistical analysis
- Returns normalized values (0.0-1.0)

[Needed ChatGPT's help, quite a bit]

`useAudioAnalyzer.js`

- Creates `AudioAnalyzer` instance
- Converts audio data to React state
- Throttles updates (60fps → 10fps)
- Handles initialization/cleanup
- Returns: `{ audioAnalyzer, audioData, init, stop, ... }`
-

`AudioContext`

- Creates React Context
- Provides hook return value to all children
- Exports `useAudio()` hook for easy consumption
- Eliminates prop drilling

Audio Analyzer

```
async init() {
  try {
    this.stream = await
navigator.mediaDevices.getUserMedia({
  audio: {
    echoCancellation: false, // We want raw audio
    noiseSuppression: false, // Don't filter
    autoGainControl: false   // Don't normalize
  }
});
} catch (error) {
  // Handle permission denied, mic in use, etc.
}
}
```

The `getUserMedia()` method of the `MediaDevices` interface prompts the user for permission to use a media input which produces a `MediaStream` with tracks containing the requested types of media.

That stream can include, for example, a video track (produced by either a hardware or virtual video source such as a camera, video recording device, screen sharing service, and so forth), an audio track (similarly, produced by a physical or virtual audio source like a microphone, A/D converter, or the like), and possibly other track types.

It returns a `Promise` that resolves to a `MediaStream` object. If the user denies permission, or matching media is not available, then the promise is rejected with `NotAllowedError` or `NotFoundError` `DOMException` respectively.

AudioContext.Provider

- makes audio state and functions available to all its child components without prop drilling.
- Any component can access via `useAudio()`

```
export function AudioProvider({ children, autoInit = true }) {  
  const audio = useAudioAnalyzer(autoInit);  
  
  return (  
    <AudioContext.Provider value={audio}>  
      {children}  
    </AudioContext.Provider>  
  );  
}
```

AudioContext.Provider

How do components consume the context?

Hook manages state

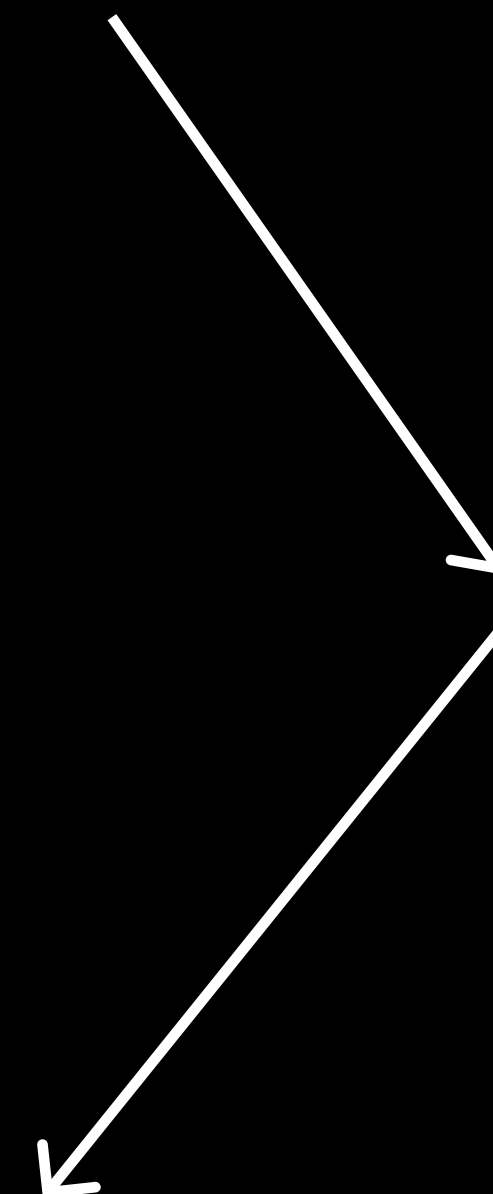
```
function useAudioAnalyzer() {  
  const [audioData, setAudioData] = useState({...});  
  // ... update logic  
  return { audioData, ... };  
}
```

Components consume

```
function SciFiFrame() {  
  const { audioData } = useAudio();  
  return <CircleSlider value={audioData.bassLevel} />;  
}
```

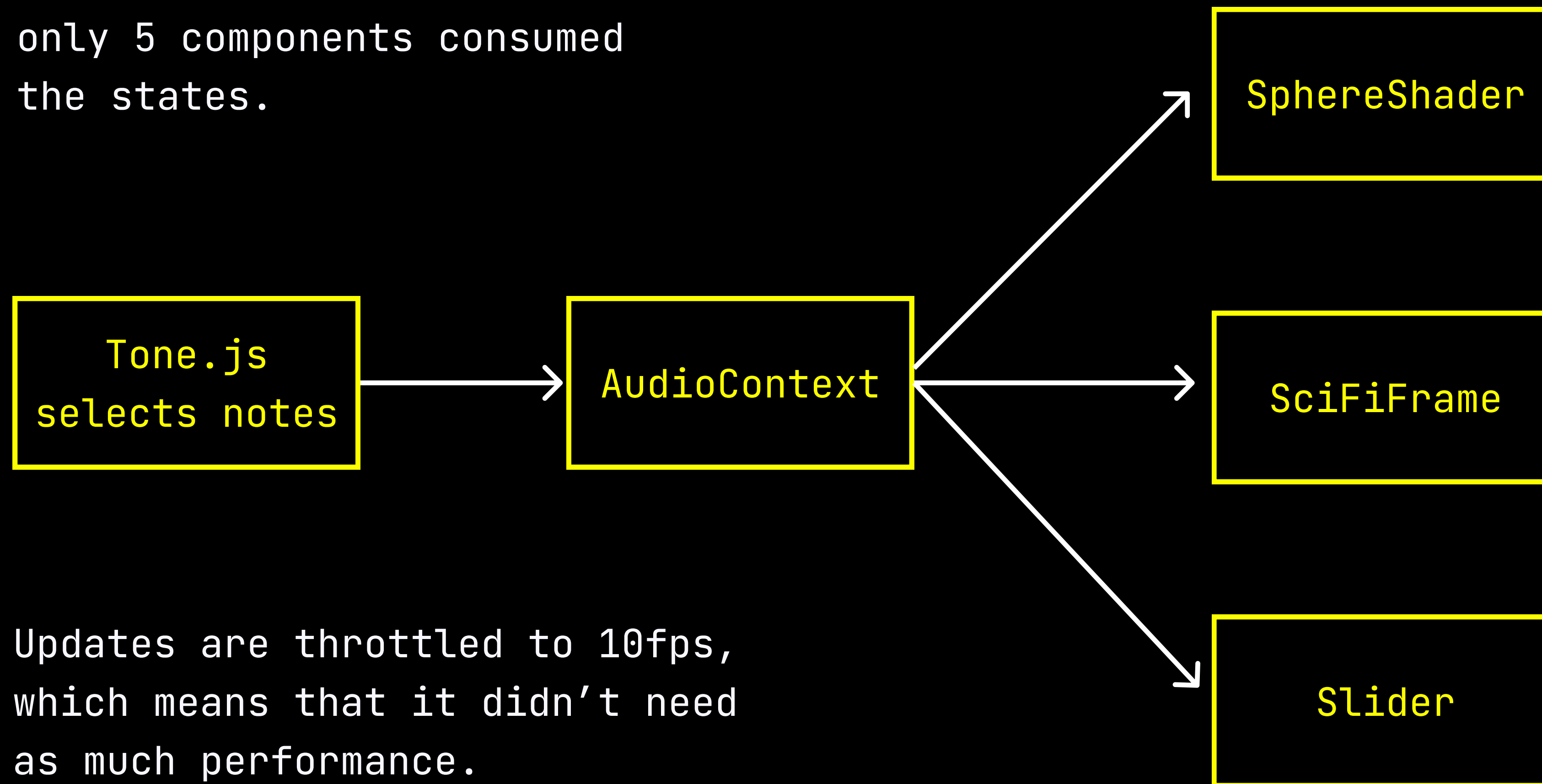
AudioProvider

```
<AudioContext.Provider  
  value={audio}>{children}</  
  AudioContext.Provider>
```



HOOKS AND NOOKS

I ended up using Context instead of Redux because only 5 components consumed the states.



Updates are throttled to 10fps, which means that it didn't need as much performance.

Redux also meant more code to maintain and more boilerplate.

My app is a SPA, did not need to pass states across multiple pages/components

CircleSlider.jsx

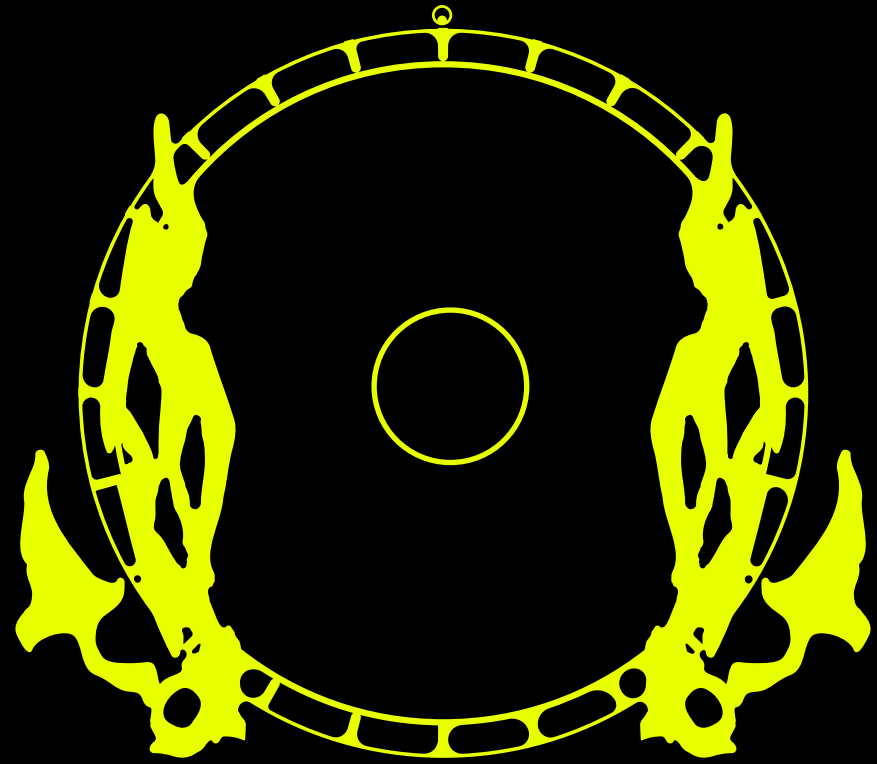


```
<circle
  className="circle-slider-progress"
  cx={size / 2}
  cy={size / 2}
  r={size / 2 - 5}
  fill="none"
  stroke="#F33833"
  strokeWidth="3"
  strokeDasharray={circumference}
  strokeDashoffset={offset}
  strokeLinecap="round"
  transform={`rotate(-90 ${size / 2} ${size / 2})`}
/>
```

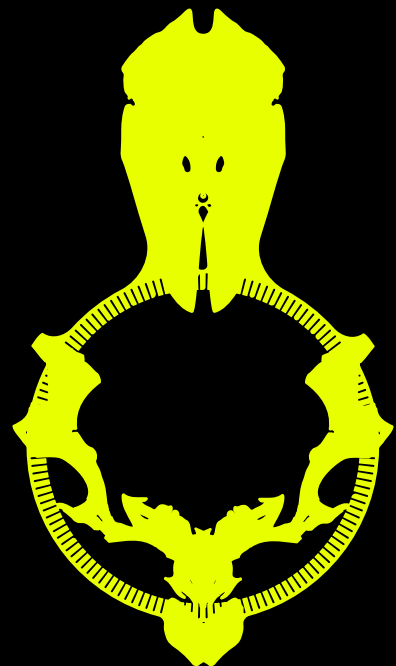
AudioVisualization.jsx



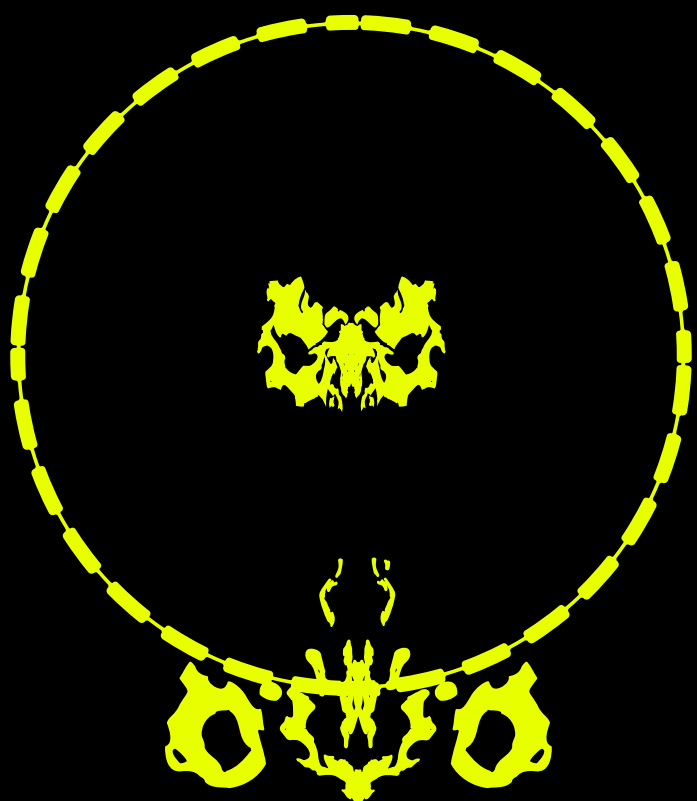
```
<div className="circle-sliders-container">
  <CircleSlider
    value={audioData.bassLevel}
    label="BASS"
    size={80}
  />
  <CircleSlider
    value={audioData.midLevel}
    label="MID"
    size={80}
  />
  <CircleSlider
    value={audioData.trebleLevel}
    label="TREBLE"
    size={80}
  />
</div>
```



high



low

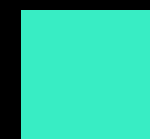


mid

colorUtils.js

The Svg colours are tied to the audio for different frequency bands.

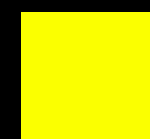
- The colours range from green to red to yellow to purple.
- Using **linear interpolation** so that when the audio value is low, it would be one color and as the audio changes, it will transition between the colours.



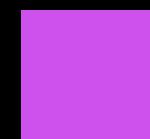
25 % - EE5252



50 % - FBFF00



75 % - CF52EE

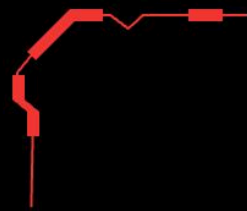


100 % - 38EEC5

```
const colors = [
  { r: 0, g: 0, b: 0 },
  { r: 238, g: 82, b: 82 },
  { r: 251, g: 255, b: 0 },
  { r: 207, g: 82, b: 238 },
  { r: 56, g: 238, b: 197 }
];
const stops = [0, 0.25, 0.5, 0.75, 1.0];

let segment = 0;
for (let i = 0; i < stops.length - 1; i++) {
  if (l ≥ stops[i] && l ≤ stops[i + 1]) {
    segment = i;
    break;
  }
}
```

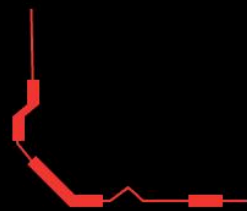
```
const r = Math.round(color1.r + (color2.r - color1.r) * smoothT);
const g = Math.round(color1.g + (color2.g - color1.g) * smoothT);
const b = Math.round(color1.b + (color2.b - color1.b) * smoothT);
```

[BASS: 43%]
[MID: 65%]
[BEAT: NO]
[MIC: ACTIVE]



COLLAPSE



SphereShader.jsx

- imports audio/AudioContext
- Used shader code from <https://codepen.io/pieceofloaf/pen/j0dobZV>

```
v.x += bassDistort * sin(z * 0.1 + float(i) * 0.5) * 0.4;  
uv.y += midDistort * cos(z * 0.15 + float(j) * 0.5) * 0.4;  
uv.x += trebleDistort * sin(z * 0.2) * 0.2;
```

Used the values from **audioContext** to distort the shader

```
if (audio) {  
  const smoothing = 0.8;  
  audioSmoothedRef.current.low =  
    audioSmoothedRef.current.low * smoothing + audio.bassLevel * (1 - smoothing);  
  audioSmoothedRef.current.mid =  
    audioSmoothedRef.current.mid * smoothing + audio.midLevel * (1 - smoothing);  
  audioSmoothedRef.current.high =  
    audioSmoothedRef.current.high * smoothing + (audio.trebleLevel || 0) * (1 - smoothing);  
  audioSmoothedRef.current.overall =  
    (audioSmoothedRef.current.low + audioSmoothedRef.current.mid + audioSmoothedRef.current.high) / 3;  
  
  uniforms.u_audioLow.value = audioSmoothedRef.current.low;  
  uniforms.u_audioMid.value = audioSmoothedRef.current.mid;  
  uniforms.u_audioHigh.value = audioSmoothedRef.current.high;  
  uniforms.u_audioOverall.value = audioSmoothedRef.current.overall;  
}
```




[three.js]shader on sphere

Mr Silver [+ Follow](#)

Settings

HTML

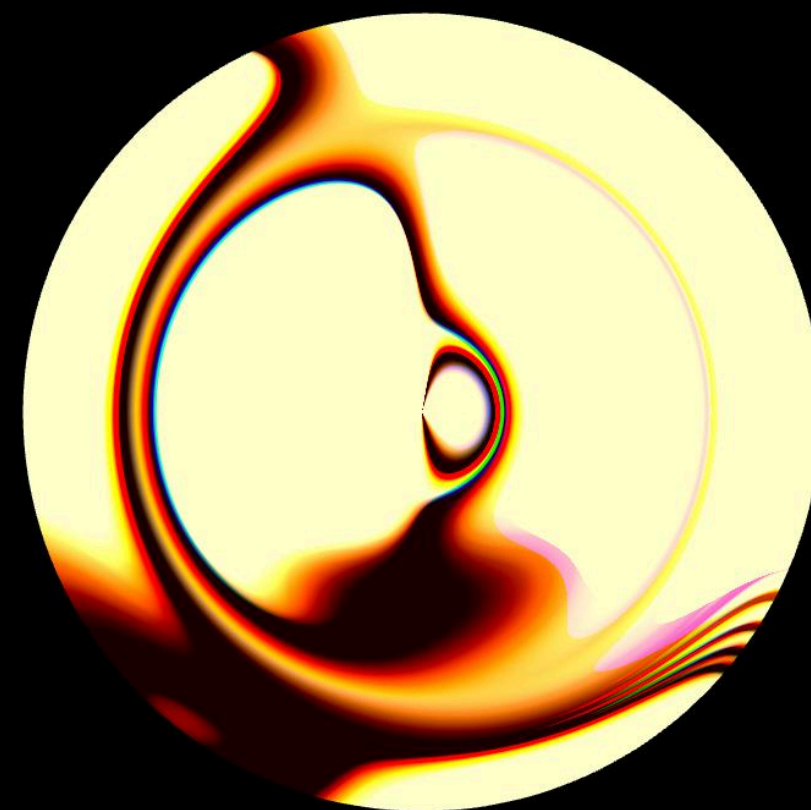
```
1 <script
  src="https://cdnjs.cloudflare.com/ajax/libs/three.js
  /110/three.min.js"></script>
2 <canvas id="myCanvas"></canvas>
3
4 <script id="fragmentShader" type="x-shader/x-
  fragment">
5   #ifdef GL_ES
6   precision mediump float;
7   #endif
8
9   #extension GL_OES_standard_derivatives : enable
10
11   uniform float time;
```

CSS

```
1 *{
2   margin: 0;
3 }
```

JS

```
14   const scene = new THREE.Scene();
15   const clock = new THREE.Clock();
16
17   camera = new THREE.PerspectiveCamera(45,
  window.innerWidth / window.innerHeight);
18   camera.position.set(0, 0, 1000);
19
20   const myGroup = new MyGroup();
21   scene.add(myGroup);
22
23   onWindowResize();
24   window.addEventListener( 'resize',
  onWindowResize, false );
25
```



Console Assets Comments Keys

Add to Collection Fork Embed Export Share



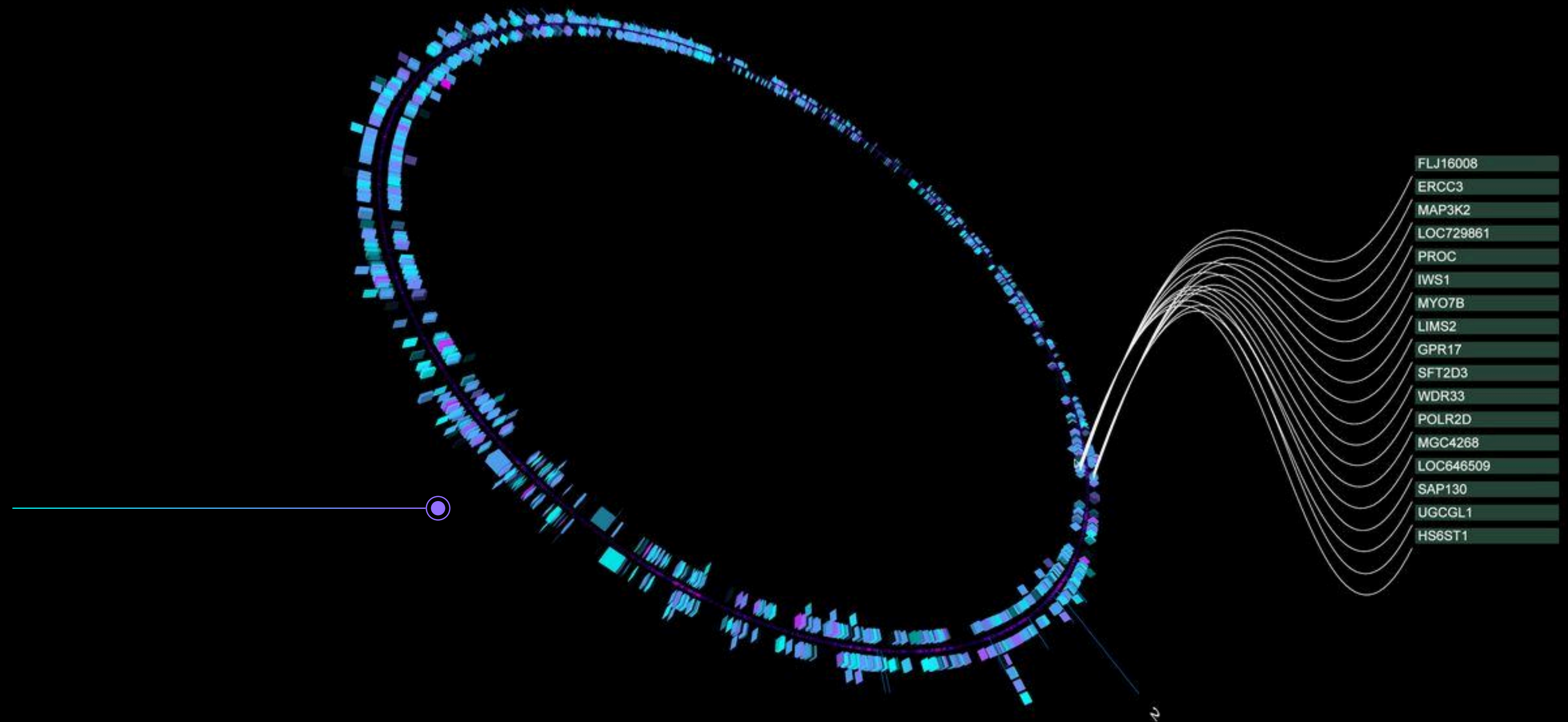
Breakdown

- concept|theme of the site
- concepts|skills we have covered that you will use to build your site
- concepts|skills you need to still learn to complete the project

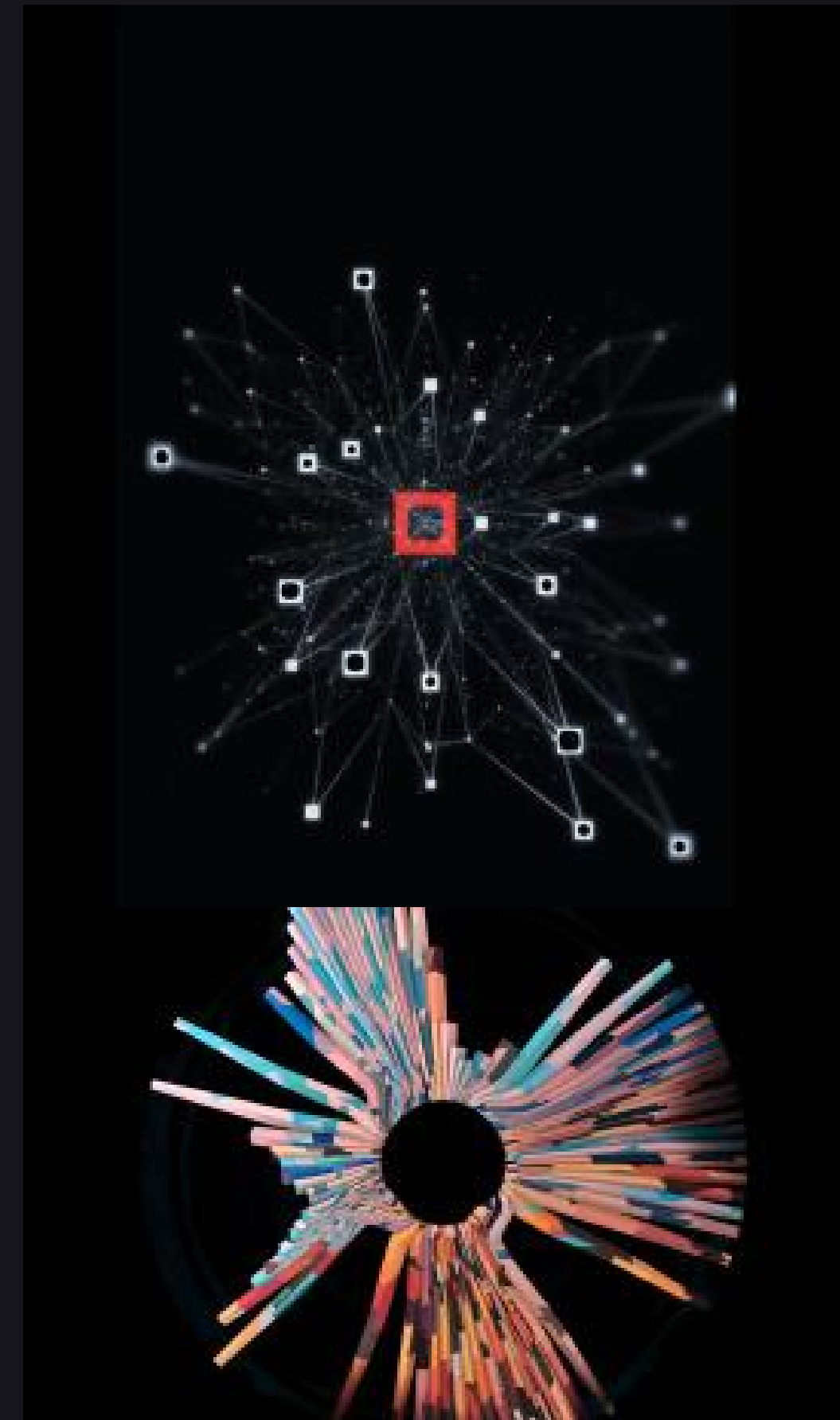
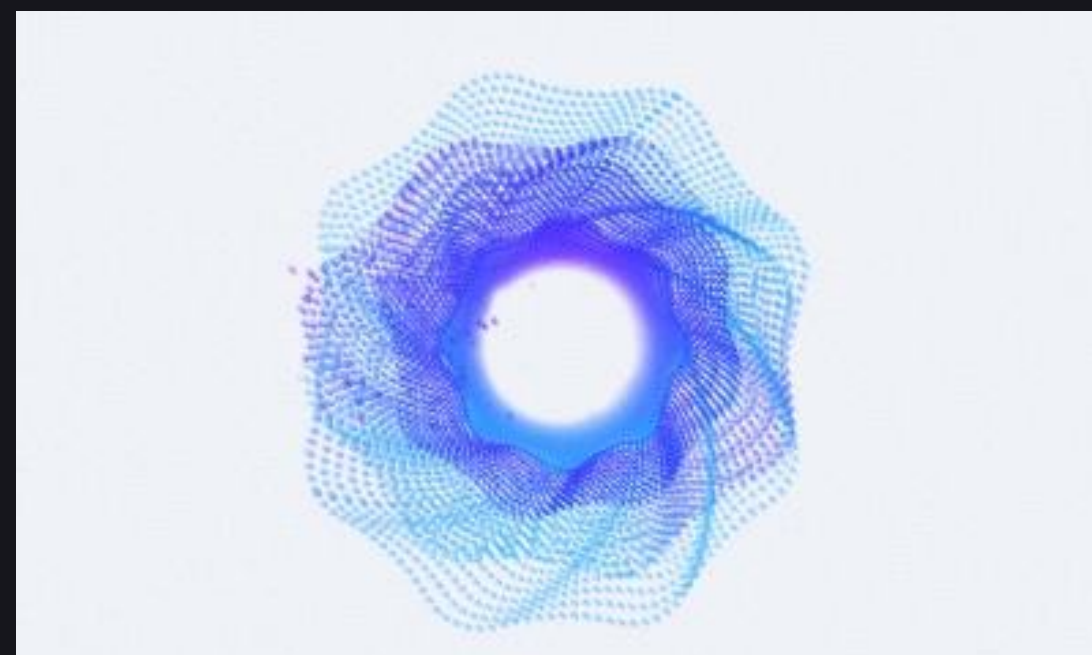
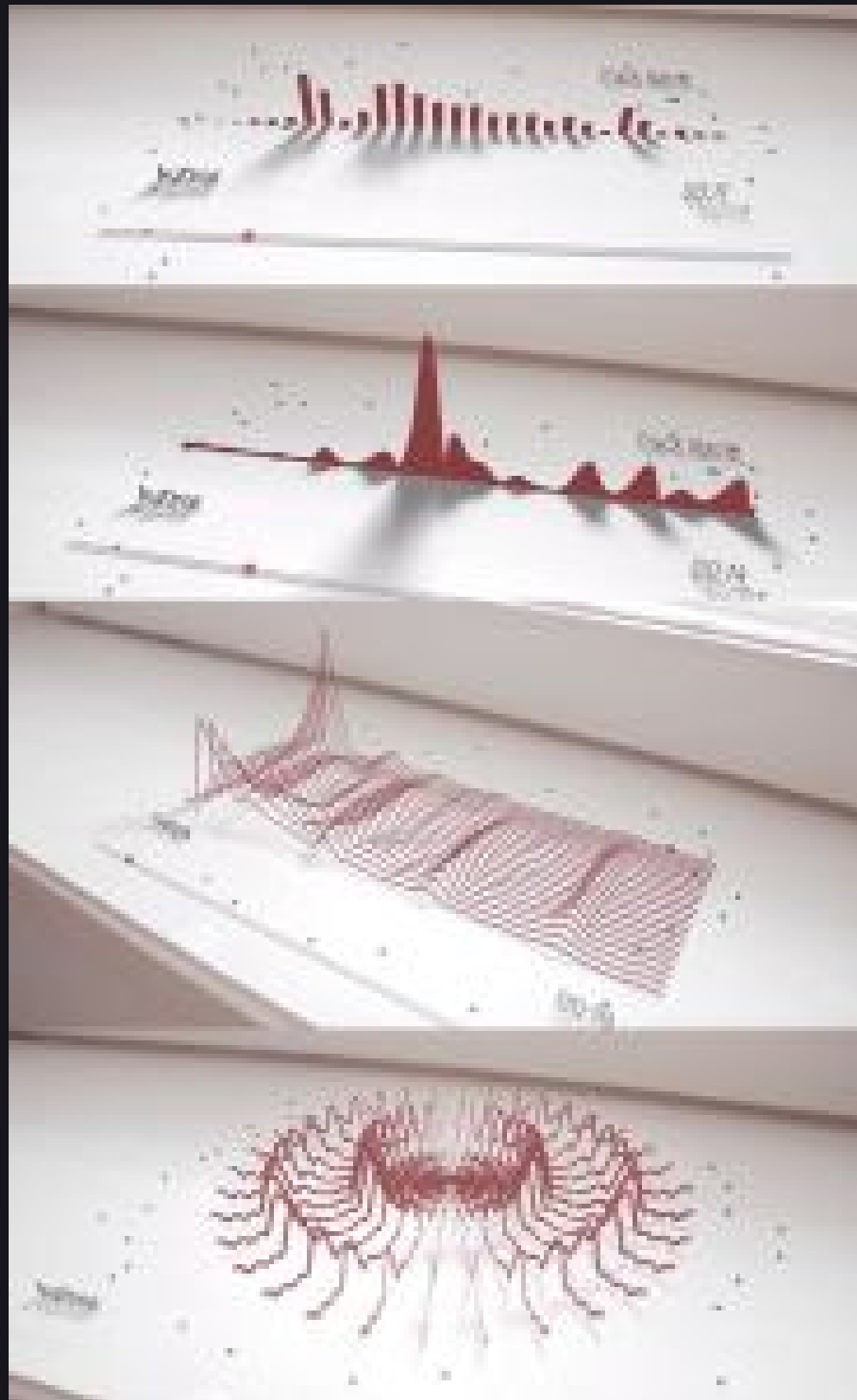
Planning

I want to create an audio
visualisation using react.
It will be a combination of
using tone.js sound, which
uses web audio API

Effects like this use a
combination of particle
effects, shaders and math
to create dynamic shapes,
but are ultimately changed
by variables/states.

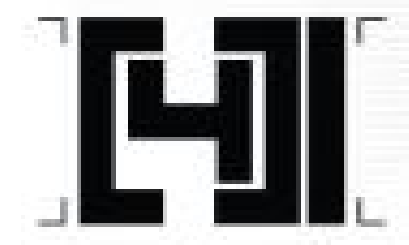


concept



I really like how we can use geometry and radial shape to evoke emotions through audio-visualisations.

[Richards Stark (2004).01]
[Richards Stark (2004).01]



DAMAGE CONTROL

While this corporate entity traditionally serves in a quarantine or repair/rebuild capacity, Damage Control also operates as the archival unit for government-contested metahuman machinery and technology that belongs to missing, incapacitated or deceased individuals where either a clear chain of custody does not exist or the items in question pose an existential threat to the United States.

Current inventory in the Damage Control Contested Storage facility is made up almost exclusively of advanced schematics and disassembled material created or owned by Reed Richards [Mister Fantastic] and Tony Stark [Iron Man].*

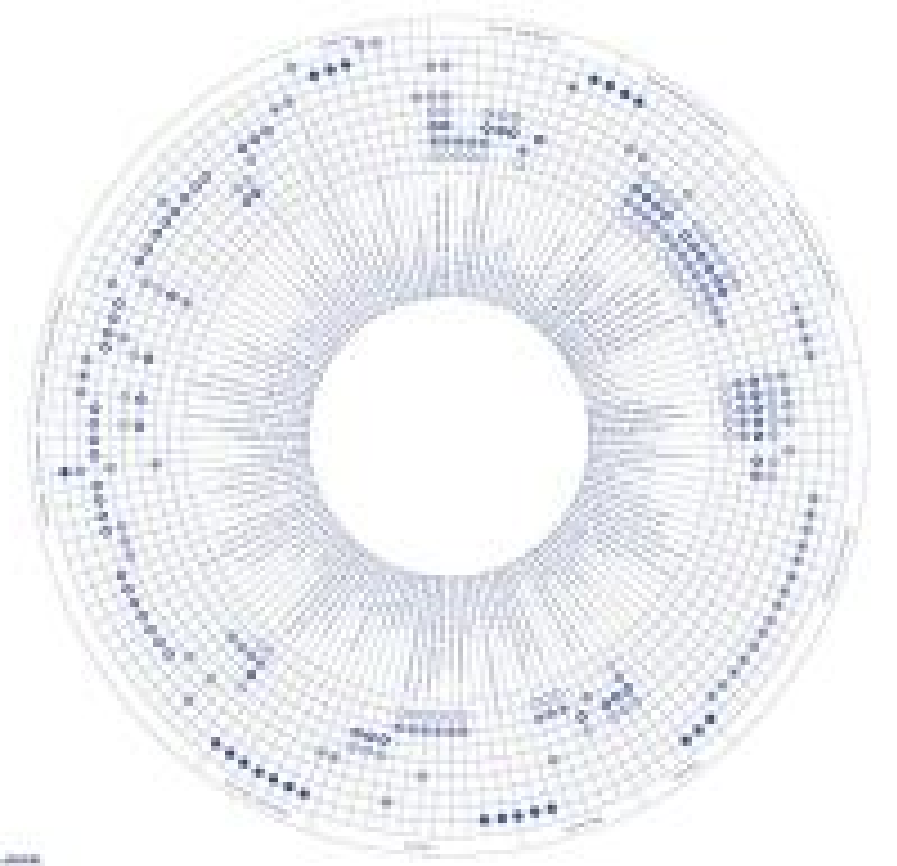
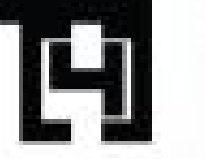
LIST OF SECURITY ITEMS (LEVEL 5)

	Sol's Arrow
	Sol's Hammer
	The Bridge
	Iron Man Mark IV
	Iron Man Mark VI
	Iron Man Mark VII
	Iron Man Mark VIII
	Warrior Mark I
	Warrior Mark II
	Antagonist Sling
	Multiversal Beacon

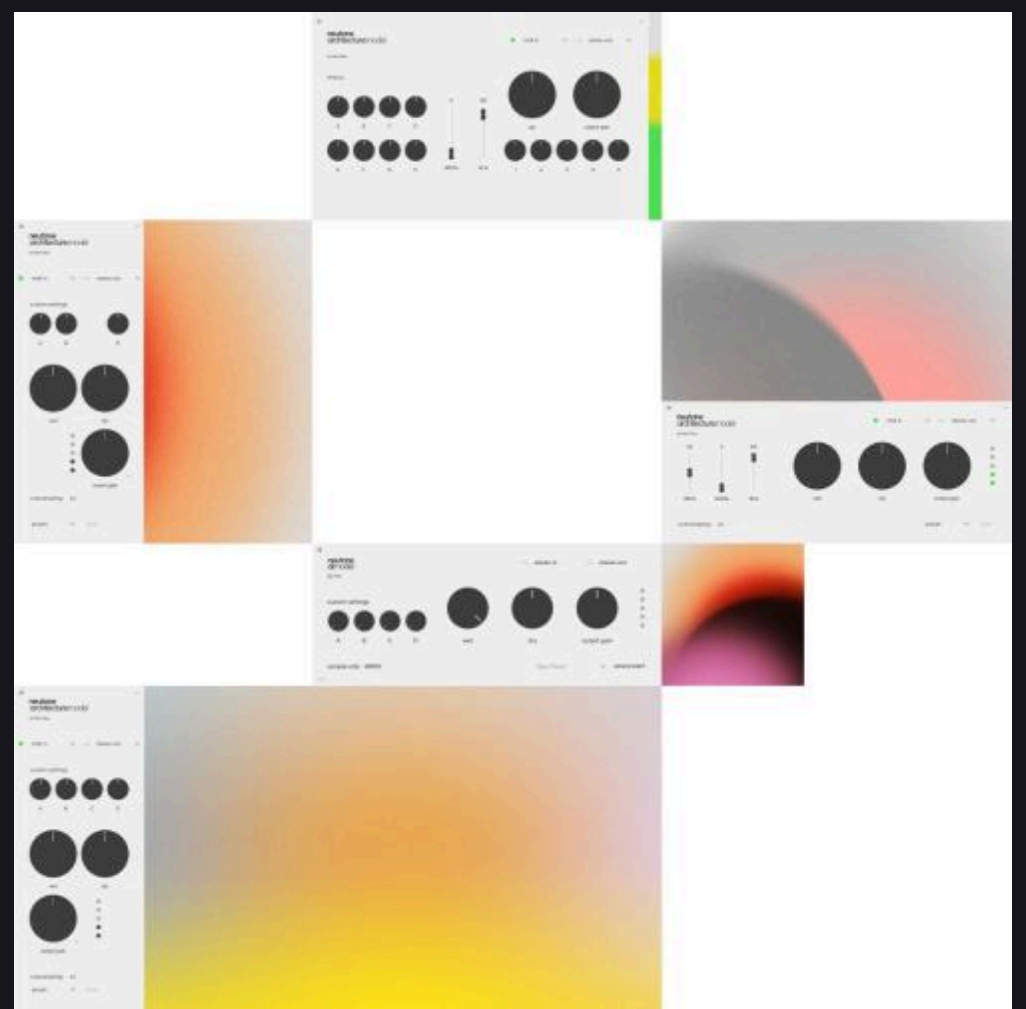
*In the recent past, both Stark and Richards have gone to great lengths to ensure that their technology did not fall into the hands of entities like Damage Control. But their normal avenue of evasion — leaving control of their intellectual property to each other — failed when Richards was missing for a prolonged period of time and Stark was presumed dead.

Rights to all current material are being contested.

[Richards Stark (2004).01]
[Richards Stark (2004).01]



- ☐ Multiversal Beacon - Antagonist Contested Storage
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist
- ☐ Multiversal Beacon - Antagonist



Visually, I'm trying to lean into clean, modern, tech aesthetics.



Exhibition

I'm also inspired by a lot of three.js projects that show how we can use shaders and distortion to create organic shapes



Fungible

HOOKS

useState

Manage local component state.

- store whether audio is playing

useRef

Reference canvas/threejs objects

useEffect

Handle side effects

- start and stop audio context

useContext

Global context for shared state

Creating parametric and procedural animations

Karbandi Pattern

Divisions (n-sided)

16-sided

Connection Distance (k)

Every 6th point

Scale

150px

Regenerate Pattern

Download as SVG

Download as PNG

Total Arches: 16

Shamseh Radius: 6.9px

Pattern Density: 37.5%

Karbandi Geometry

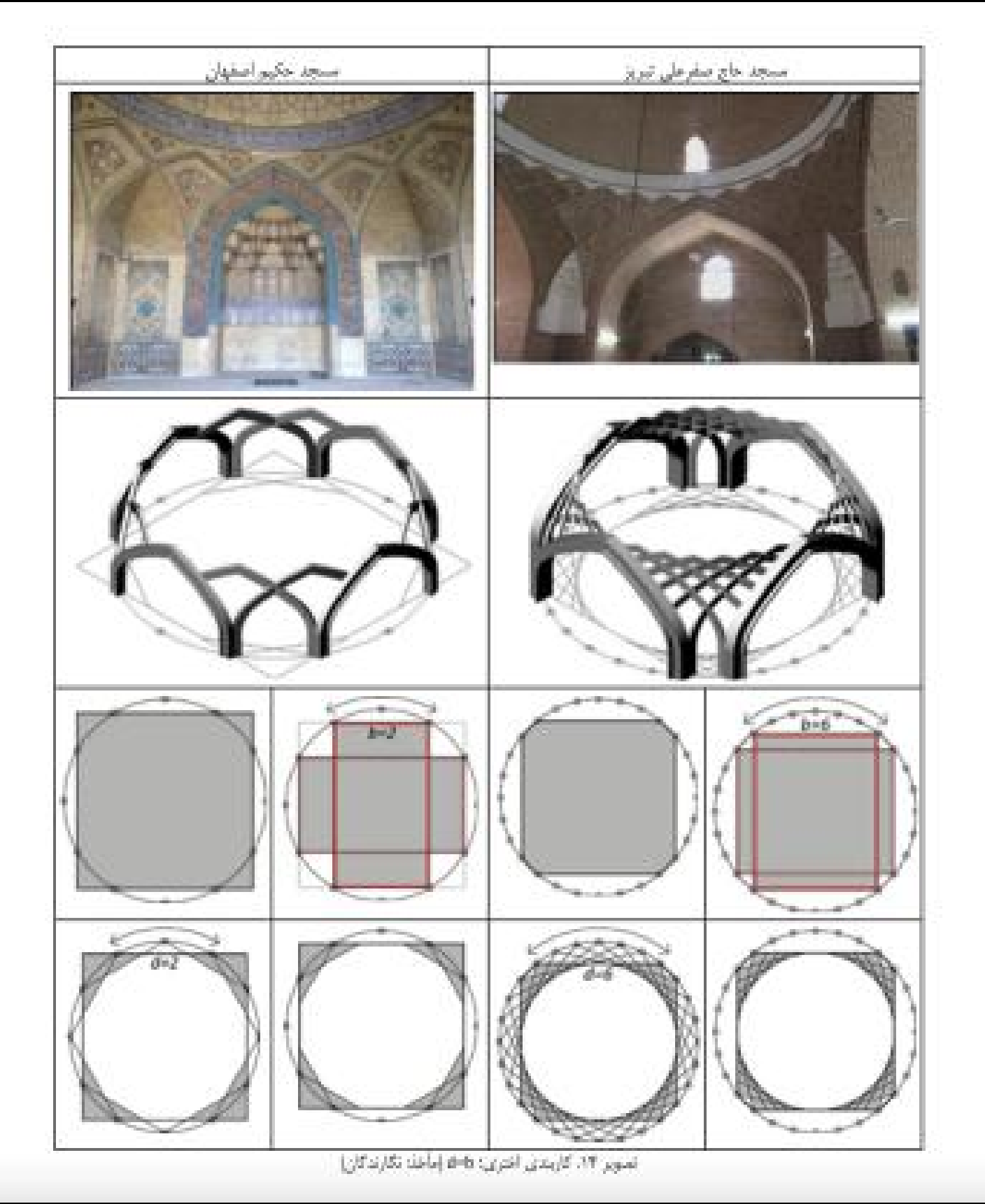
This 2D visualization shows the geometric pattern from Persian architecture. The pattern is based on dividing a circle and connecting points at regular intervals.

Note: For 3D visualization, download the files and open them in a browser on your local machine (not in a sandboxed environment).

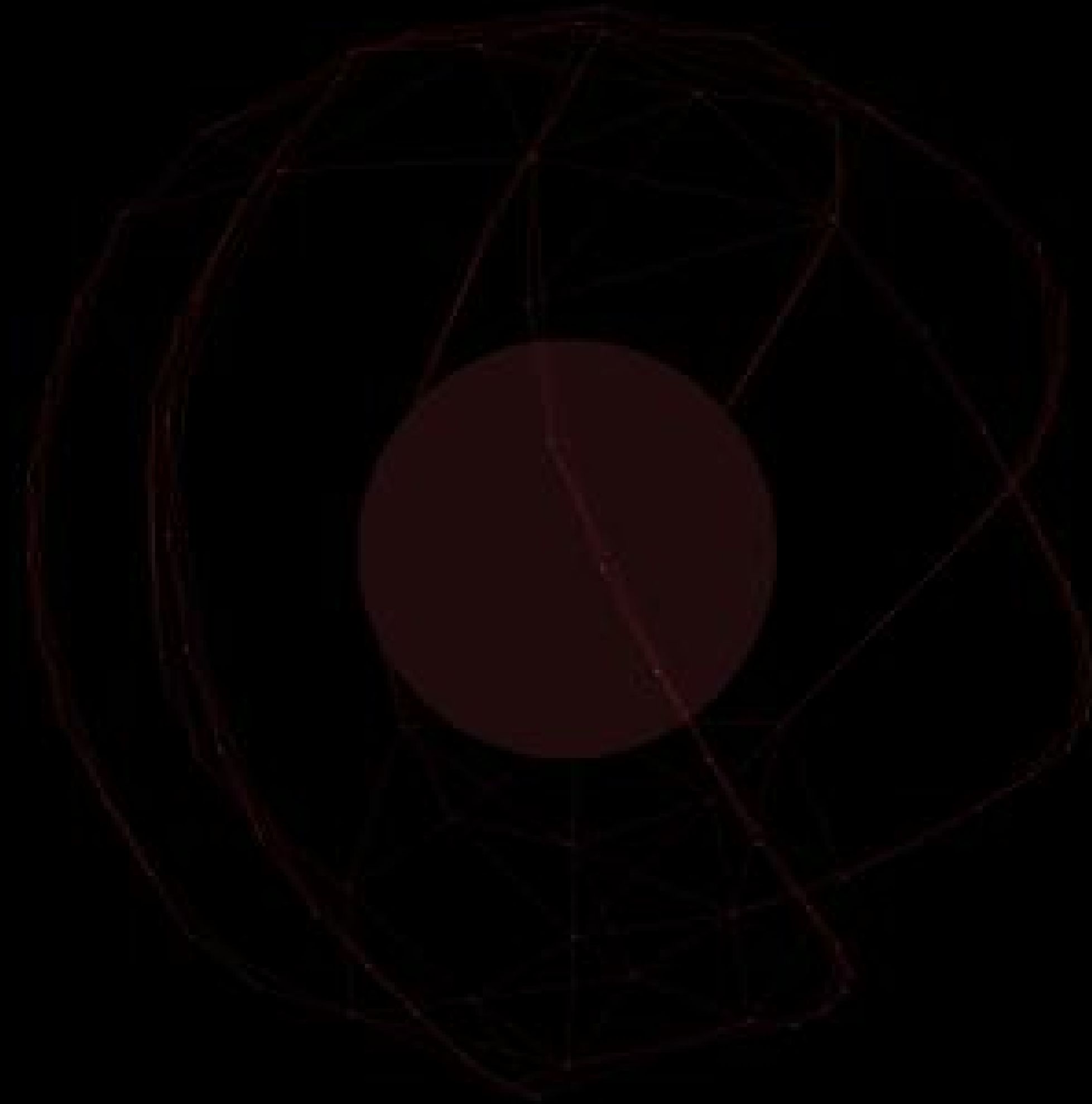
Plan View

Elevation View

Pattern Only



How do you
create this
effect



Step 1: Base Sphere Distribution (Fibonacci Algorithm)

```
const phi = Math.PI * (3 - Math.sqrt(5)) // Golden angle ≈  
2.39996 radians  
const y = 1 - (index / (total - 1)) * 2 // Y coordinate:  
-1 to 1  
const radius = Math.sqrt(1 - y * y) // Radius at this  
Y level  
const theta = phi * index // Angle around  
Y-axis  
  
x = Math.cos(theta) * radius  
y = y  
z = Math.sin(theta) * radius
```

